Note: how Scribble Room syncs, orders, recovers and scales

1. How the canvas is synced and how stroke data is sent

The picture is not sent as pixels. It is a **stack of actions**: strokes, fills and clears. The drawer's browser sends small operations over the WebSocket:

- `ds` starts a stroke (id, tool, colour, size, first point), `dp` adds points, `de` ends it.
- `fill` (point and colour), `clear`, and `undo` (which pops the last action).

Coordinates are integers on a fixed 800 x 600 logical canvas, so the same message draws the same thing on any screen size. Points are collected while the pointer moves and sent **once per animation frame (about 60 times a second), at most 64 points per message**, so a fast stroke is a handful of small messages instead of hundreds. The server checks every field (ids, colours, ranges, sizes, counts), applies the operation to its own copy of the action stack, and relays it to everyone else. Because the server holds the stack, a **late joiner or a reconnecting player gets the whole stack in one message** and re-renders it. Undo is simple for the same reason: pop the action, and every client re-renders from the stack. The server also keeps a timestamped log of every operation, which is what the end-of-game replay plays back.

2. Latency and ordering

- **Ordering:** there is a single writer (the current drawer) and one TCP-ordered WebSocket per player, so operations cannot arrive out of order. The server still stamps each operation with a sequence number per turn. A client that sees a gap (for example after a reconnect) asks for a full snapshot (`resync`) instead of drawing a corrupted picture.
- **No wait for the server on the drawer's side:** the drawer's browser draws immediately and the server does not echo the operation back to the drawer, so drawing always feels instant. If the server ever refuses one of the drawer's operations (rate or size limit), it sends the drawer its own copy of the canvas, so the drawer's optimistic picture can never drift from what everyone else sees.
- **Steady send rate on any screen:** batches go out once per animation frame but at least 16 ms apart, so a 144 Hz monitor sends the same ~60 messages a second as a 60 Hz one and stays well inside the rate limit.
- **Low delay for everyone else:** no message compression (it costs more CPU than it saves on tiny messages), `TCP_NODELAY`, one batch per frame, and immediate relay on the server.
- **Clocks:** timers and the "who is first to guess" decision exist only on the server. Browsers only display the countdown, using an offset measured by a ping every 4 seconds.
- **Measured** (localhost, so excluding real internet latency): p50 stroke delivery of 1 ms and p99 of 4 ms at 100 rooms (see section 4).

3. What happens when the server restarts

- On `SIGTERM`/`SIGINT` (and `SIGBREAK`/`SIGHUP` or an IPC message on Windows, which has no catchable `SIGTERM`) the server writes a **snapshot of every room** (players and their secret tokens, scores, the current word and canvas, the turn clock, replay logs). The file is written to a temporary name and renamed, so a crash never leaves a half-written file. While running, it also snapshots about every 2 seconds when something changed, so even a hard crash loses at most the last couple of seconds.
- On start it **restores the rooms**. Every player starts as "disconnected" with the usual 30-second grace period, and the turn clock resumes with the time that was left, because downtime does not count against the drawer.
- Browsers **reconnect automatically** (backoff from 0.3 to 3 seconds) and rejoin with their token, so each player gets the same seat, the same score and the same canvas. A banner says "Reconnecting" meanwhile.
- This is covered by an automated test (kill with `SIGTERM`, restart, rejoin, check word, canvas, score and clock).
- Limits: strokes drawn in the last ~2 seconds before a hard crash can be lost. With more than one server instance, the snapshot would have to move to a shared store such as Redis, with each room pinned to one instance.

4. How many rooms one server instance can handle, and how it was measured

I wrote a load test (`loadtest/loadtest.js`) that starts a real server and fills it with rooms of 4 bot players each. In every room the drawer holds the pen down for the whole test and streams **30 messages a second (4 points each)**, which is heavier than real play because real drawers lift the pen and pause. Guessers send a chat line every few seconds. All bots share one clock, so stroke latency (send to receive) is exact. CPU, event-loop delay and memory come from the server's own `/api/metrics`. Each row is 15 seconds of measurement, and the bots run on the same machine as the server.

Linux, 1 vCPU shared with the bots (`loadtest/results-linux.json`):

Rooms	Sockets	Stroke latency p50 / p95 / p99	Server CPU	Event-loop delay p99	Memory
100	400	1 / 2 / 4 ms	11.7%	15 ms	113 MB
300	1,200	1 / 8 / 32 ms	27.9%	34 ms	156 MB
500	2,000	4 / 26 / 52 ms	43.8%	42 ms	221 MB
600	2,400	14 / 55 / 116 ms	47.6%	222 ms	242 MB

Windows 11 laptop, i5-13450HX (`loadtest/results-windows.json`):

Rooms	Sockets	Stroke latency p50 / p95 / p99	Server CPU	Event-loop delay p99	Memory
100	400	8 / 19 / 26 ms	49%	35 ms	95 MB
200	800	12 / 24 / 36 ms	77%	37 ms	111 MB
250	1,000	42 / 92 / 112 ms	97%	70 ms	135 MB
300	1,200	621 / 1,412 / 1,532 ms	97%	640 ms	211 MB

Answer: about 500 rooms (2,000 players) per process on Linux, and about 200 rooms on a Windows laptop, in the worst case where every drawer draws non-stop. Past that point the tail latency climbs above 100 ms and then the event loop falls behind. Memory is not the limit: roughly 0.3 to 0.5 MB per room.

Where the time goes. I profiled the server at 200 rooms (`node --cpu-prof`). About 70% of its busy time is the `writev` system call, meaning one kernel write per relayed message per recipient. Parsing, validation and the game logic together are about 5%. So capacity is set by how many small messages per second the operating system can write (about 27,000 a second used 28% of a core on Linux and all of a core on Windows loopback), not by the game code. That is also why the Linux numbers are so much better.

Measuring on Windows. Windows tends to put a background process without a window on the efficiency cores of a hybrid CPU, which made the first runs swing wildly (one run showed seconds of delay at 200 rooms while the server reported only 69% CPU). The load test therefore runs the server at high priority, the way a real server process would run. `LOADTEST_NORMAL_PRIORITY=1` turns that off.

Scaling further. The next step would be to cut the number of writes: relay a room's strokes on a short shared tick (for example every 30 ms), so that several stroke messages go out as one write per recipient, at the cost of a little latency. Beyond one core, run one process per core and route each room code to one process (rooms never talk to each other, so this splits cleanly), with the snapshots in a shared store such as Redis. Rerun the test with `npm run loadtest -- --steps 100,200,300,500`.